Support for Exercise 2. Rule-based and data-driven examples

1. For reading the annotation files into dataframes (DF) use index_col = False argument. Otherwise the first column (date and time stamp) will be read as an index, complicating further data manipulation.
2. Pandas library allows filtering the content of a DF by focusing on one/various values of one/various columns. To get the resultant filtered dataset according to the specific criteria use DataFrame.loc function, below you can see an example:

```
data = {'Column1': [3, 2, 3, 4, 3],
        'Column2': ['A', 'B', 'C', 'D', 'A']}
df = pd.DataFrame(data)

# Filter the DataFrame where Column1 is equal to 3
filtered_df = df.loc[df['Column1'] == 3]
```

3. It is recommended to merge the 3 filtered DF (one for each cardiac failure condition) that should have been generated in task 1, and then, sort the resultant DF by 'Time' variable/column by using 'sort_values' method.
   Pandas also allows checking multiple conditions across the columns at the same time and generate and additional Boolean column with the results as follows:

```
df['condition1']=(df['Column1'] < 3)&(df['Column2'] =='A')
```

The following code might be beneficial plot the presence of cardiac failure correctly.

```
df_rules['Time_formatted']=pd.to_datetime((df_rules['Time']),format='%d%H%M%S')

import matplotlib.pyplot as plt

import matplotlib.dates as mdates

date_form = mdates.DateFormatter("%H:%M")

fig, ax = plt.subplots(figsize = (12, 8))

ax.scatter(df_rules['Time_formatted'], df_rules['cardiac_failure'])

ax.xaxis.set_major_formatter(date_form)

plt.xlabel('time (HH:MM)')

plt.ylabel('Cardiac failure (1-True, 0-False)')
```

4. The time has to be handled in the same way as with AN1 dataset and proceed similarly with the condition check of task 2 considering the info in Appendix B.